

Claude Managed Agents

A Conceptual Tutorial

Jamie Jaworski
and Claude Opus 4.7

April 2026

Contents

About this document	4
1. What Claude Managed Agents is, and what problem it solves	5
1.1 The architectural shift	5
1.2 Where Managed Agents sits relative to other Claude products	6
1.3 What this means for a security architect	6
2. The four core concepts, in depth	8
2.1 Agent	9
2.2 Environment	9
2.3 Session	10
2.4 Events	10
3. How it actually works: the lifecycle of a session	12
3.1 Phase 1 — Create an Agent	12
3.2 Phase 2 — Create an Environment	13
3.3 Phase 3 — Start a Session	13
3.4 Phase 4 — Send events and stream responses	13
3.5 Phase 5 — Steer or interrupt	14
3.6 Tying it back to the conceptual model	14
4. A walkthrough of the interface	15
4.1 The ant CLI	15
4.2 The SDK	16
4.3 The Claude Console	16
4.4 Choosing between them	18
5. Getting started: prerequisites and first agent	19
5.1 Prerequisites	19
5.2 Installing the tools	19
5.3 A minimal first agent	20
6. Building and deploying a CVE monitor end-to-end	23
6.1 The agent definition	23
6.2 The environment definition	24
6.3 What the agent does, end to end	25
6.4 The invocation script	26
6.5 Deploying for repeated invocation	28
6.6 What you have built, and what comes next	30
7. The security architect's lens	31
7.1 What moved to Anthropic's trust boundary	31

7.2 What stayed firmly with you	33
7.3 The new attack surfaces	34
7.4 A defense-in-depth checklist for Managed Agents deployments	36
7.5 Where the threat model is still genuinely open	38
8. Where Managed Agents fits in your toolbox	39
8.1 Where it fits well	39
8.2 Where it fits poorly	40
8.3 What to reach for instead	40
8.4 A closing thought for the security architect	41
Appendices	42
Appendix A — Reference cards for the four abstractions.....	42
Appendix B — Glossary	46
Appendix C — Further reading.....	48
Appendix D — Terms and abbreviations.....	50
Appendix E — Harness: an extended definition.....	52

About this document

This is a tutorial on Claude Managed Agents — a service Anthropic launched in public beta on April 8, 2026, that runs Claude as an autonomous agent in managed cloud infrastructure. It is written for a security architect who is comfortable with general agent concepts but new to this specific product, and who prefers conceptual depth over implementation minutiae.

The document is organized in eight sections plus appendices. Sections 1 through 3 are the conceptual spine: what Managed Agents is and the problem it solves, the four core abstractions on which the product is built, and the lifecycle of a session. Sections 4 through 6 are the practical walkthrough: a tour of the three interfaces, prerequisites and a first agent, and a hands-on example built end-to-end. Sections 7 and 8 are the security architect's lens and a closing decision-tree for when to use the platform at all. Five appendices follow: reference cards for the four abstractions, a glossary, a further-reading list, a quick-reference table of terms and abbreviations, and an extended definition of the term "harness."

The document is written to be read end to end, but the appendices are designed to stand alone for later reference. Section numbers and figure numbers are stable across the whole document; cross-references inside the document point to them by number.

A note on currency

Claude Managed Agents is in public beta. Specifics — the exact tool list, the beta header, rate limits, pricing — may shift between releases. This tutorial is oriented toward the durable concepts (the abstractions, the trust boundaries, the architectural choices) rather than the API surface. Where API details appear, they reflect the state of the product in April 2026 and should be re-checked against the official documentation before being relied upon.

1. What Claude Managed Agents is, and what problem it solves

Until recently, building a production agent on top of Claude meant building most of an operating environment yourself. You wrote the agent loop, you provisioned the sandbox, you wired up tool execution, you handled credentials, you managed session state, you instrumented for tracing, and you reinvented prompt caching and context compaction as your sessions grew long enough to bump into context window limits. The model was the easy part. The plumbing took months.

Claude Managed Agents is Anthropic's answer to that plumbing. It is a hosted service on the Claude Platform that runs long-horizon agents on your behalf, exposing a small set of stable interfaces — Agents, Environments, Sessions, and Events — that sit above whatever specific harness Anthropic happens to be running underneath at any given time. You define what the agent is and what it should do; Anthropic operates the execution.

1.1 The architectural shift

In the agent architecture you and I have walked through previously, an agent was an assembly of six pieces: an LLM, an agent loop, a set of tools, a memory layer, a context window, and a sandbox environment. In a do-it-yourself implementation — the kind you build against the Messages API — you own and operate every one of those pieces except the LLM itself.

Managed Agents inverts that ownership model. The agent loop, the sandbox, the tool execution layer, session storage, credential handling, and the optimizations that make long-running sessions feasible (prompt caching, compaction, checkpointing) all move into Anthropic's infrastructure. What stays with you is the agent's definition (model choice, system prompt, tool surface, MCP servers, skills), the user goal you submit, the events you stream back, and any application-level logic you build on top.

Anthropic's engineering team describes this as a "meta-harness": a system general enough to host many possible specific harnesses, much as an operating system is general enough to host many possible programs. The intent is for the four interfaces — Agent, Environment, Session, Events — to outlast the particular harness implementation Anthropic is running today, in the same way that the `read()` system call has outlasted decades of evolving disk hardware. Whether that promise holds in practice is a question for time, but the architectural ambition is worth understanding because it shapes every design decision in the product.

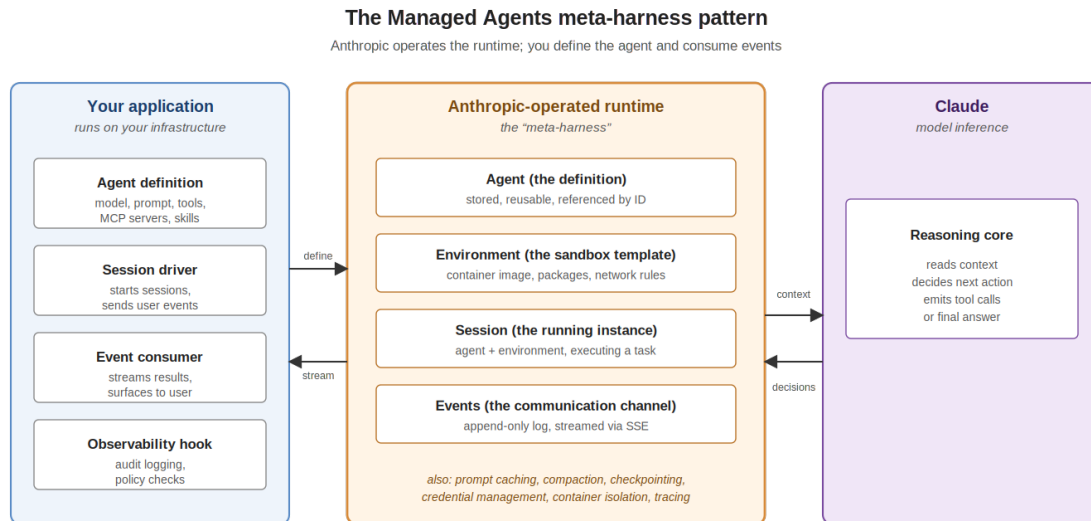


Figure 1.1 — The meta-harness pattern. Your application defines the agent and consumes the event stream; Anthropic operates everything in between.

1.2 Where Managed Agents sits relative to other Claude products

It is useful to place Managed Agents alongside the other ways you can build with Claude, because each occupies a different point on the trade-off between control and convenience.

- **Messages API.** Direct model prompting. You build your own loop, your own tool execution, your own everything. Maximum control, maximum responsibility. Best when you need fine-grained control over context assembly, latency, or data flow — or when your security or compliance posture requires that the agent loop run inside your own infrastructure.
- **Claude Code.** A specialist coding harness with its own tool set, optimized for software development. Not a general-purpose agent platform, but an excellent example of a task-specific harness.
- **Claude Managed Agents.** A general-purpose, long-running, sandboxed agent runtime that you configure rather than build. Optimized for tasks that run for minutes or hours, span many tool calls, and need to survive network disconnections.

The decision between the Messages API and Managed Agents is not really a decision about which is better. It is a decision about whose infrastructure you want to be running. If your problem is a short, interactive turn — a chat completion, a one-shot generation — you do not need Managed Agents and would not benefit from it. If your problem is "go off and do something for an hour, calling tools as you go, and tell me when you are done," you almost certainly want Managed Agents unless you have a specific reason to keep the runtime in-house.

1.3 What this means for a security architect

The architectural shift in Managed Agents is also a trust shift, and it deserves to be named explicitly up front. Several things that used to be your problem are now Anthropic's problem: container isolation, sandbox escape, the integrity of the agent loop, the durability and confidentiality of session storage, the management of tool execution credentials. Several things remain firmly your problem: which tools the agent has, what is in its system prompt, what MCP servers it talks to, what content enters its event stream, and what your application does with the outputs.

The new interfaces also create new kinds of attack surface that did not exist when you owned the loop. The event log is now the central artifact — it is what the model reads, what your application reads, what is persisted server-side, and what (in research-preview multi-agent configurations) other agents read. Anything that crosses that surface, from tool result to MCP message to retrieved web content, lives in a single shared substrate. We will return to this in Section 7. For now, the point is that adopting Managed Agents is not just a convenience choice; it is a redrawing of the trust model, and the redrawing is worth doing deliberately.

2. The four core concepts, in depth

Managed Agents is built around four abstractions: Agent, Environment, Session, and Events. They are simple individually — the design intent is that each could be explained on a single page — but the way they compose is where the leverage lives. Understanding them clearly makes the rest of the product fall into place; misunderstanding them produces the kind of confused threat models that are dangerous in a security context.

Two distinctions matter before we go into each one.

The first is between definitions and instances. An Agent is a definition; a Session is an instance. The same Agent can be the basis for many concurrent Sessions, the way the same class can be the basis for many concurrent objects. An Environment is also a definition (a container template); the actual running container is part of a Session. This matters because the security properties of definitions and instances are different. A definition is a configuration object — you can review it, sign it, version it. An instance is a runtime object — you can monitor it, isolate it, terminate it. The defenses that apply to each are different.

The second is between configuration and content. The Agent and Environment carry your configuration: deliberate, vetted, slow-changing. The Session carries content: the live conversation, tool outputs, retrieved data, user input — fast-moving, high-volume, and to varying degrees untrusted. The architectural lines between these four concepts are also lines between trusted configuration and untrusted content, and most of the security analysis later in this tutorial is a careful tracing of where those lines run.

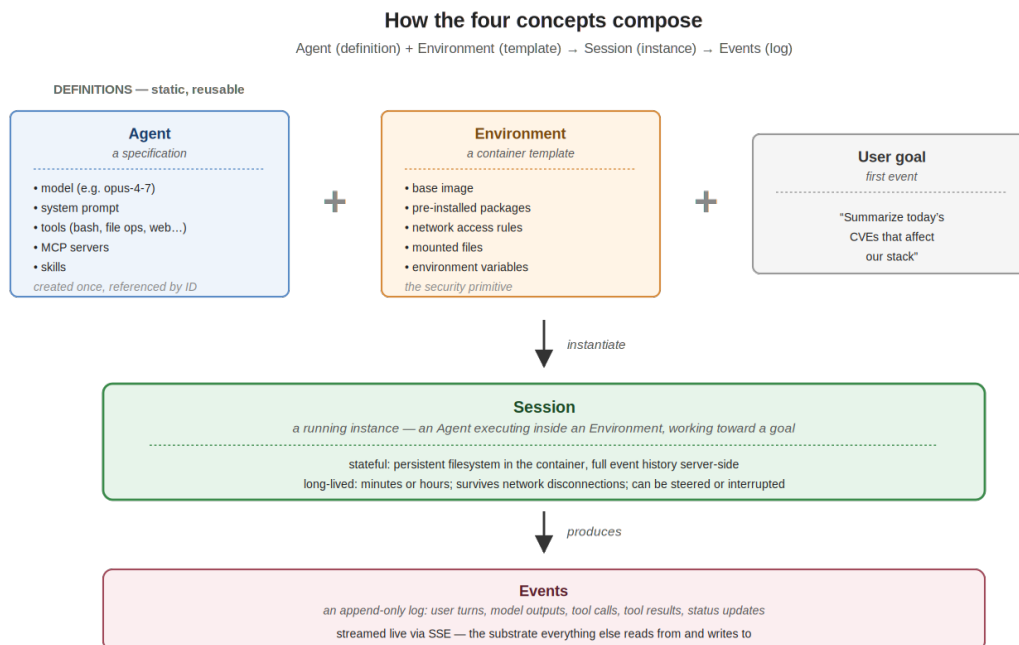


Figure 2.1 — How the four concepts compose. An Agent definition and an Environment template are combined with a user goal to instantiate a Session, which produces an append-only event log.

2.1 Agent

An Agent is a definition. It is a configuration object that says: when you run me, use this model, with this system prompt, with these tools available, talking to these MCP servers, with these skills loaded. It is created once via a single API call and is then referenced by ID across every Session it participates in.

Concretely, an Agent specifies:

- **Model.** Which Claude model performs the inference — typically Opus or Sonnet for production agents.
- **System prompt.** The role, rules, constraints, and goal-shaping instructions that prefix every conversation.
- **Tools.** Either an enabled bundle of pre-built tools (bash, file operations, web search and fetch, code execution) or a curated subset thereof.
- **MCP servers.** External tool providers the agent can call — your own services, third-party integrations, internal APIs.
- **Skills.** Reusable capability bundles (instructions plus optional code and resources) that extend what the agent knows how to do without bloating the system prompt.

The Agent is, in effect, the persona and capability surface of the agent. It is also the place where most of your offensive-security relevant decisions live: which tools to enable, which MCP servers to trust, what the system prompt forbids and permits. Because Agents are reusable and referenced by ID, you can also subject them to the kind of change-management you would apply to any other security-relevant configuration: review, version, sign, gate behind approval.

Agents do not run. They are inert until a Session brings them to life inside an Environment.

2.2 Environment

An Environment is also a definition, but a different kind. Where an Agent describes the agent's mind — what it knows and what it can do — an Environment describes its body: the container in which it executes. It is a template that, when a Session is started, is used to provision an actual sandbox.

An Environment specifies:

- **Base image and pre-installed packages.** Python, Node.js, Go, system utilities, language-specific libraries.
- **Network access rules.** What the container can reach. This is the egress control point.
- **Mounted files.** Static assets the agent should have available — reference data, configuration files, snapshots of inventories that the agent will reason over.

- **Environment variables.** Values the agent's tools may reference, including (with appropriate care) credentials.

The Environment is the security primitive in Managed Agents. It is where the runtime's "least privilege" controls actually live: not in what the agent says, but in what the agent's container is physically able to do. A poorly configured system prompt can be talked around by a sufficiently determined attacker; a network policy that prevents egress to anywhere except a specific allowlist is much harder to talk around because the LLM does not get a vote on it.

A consequence of this is that two agents that look identical at the model level can have radically different security postures depending on the Environment they run in. The same vulnerability-monitoring agent could be deployed in an Environment with broad internet access (high capability, high blast radius) or in an Environment with egress allowed only to a specific CVE feed (lower capability, much smaller blast radius). The choice of Environment is at least as load-bearing as the choice of system prompt, and arguably more so.

2.3 Session

A Session is the running thing. It is what gets created when you say: take this Agent, instantiate it inside this Environment, give it this user goal, and let it work. A Session has identity, state, and lifetime. It can be queried, streamed, steered, interrupted, and resumed. Its filesystem persists. Its event history is durably stored.

Three properties of Sessions are worth dwelling on, because they shape both the capabilities and the risks of the platform.

Statefulness. Within a Session, the container has a persistent filesystem. The agent can write a file, run something else, come back, and read it again. This is what makes long-horizon work possible: the agent can accumulate intermediate results in the filesystem rather than carrying everything in its context window.

Durability. The event log is server-side. If the network drops between your application and Anthropic, the Session keeps running. When you reconnect, you can pick up the stream where you left off. This decouples the lifetime of the agent from the lifetime of any one client connection.

Steerability. A Session is not fire-and-forget. While it is running, you can send additional user events into the event stream to redirect it, narrow its focus, answer a clarifying question, or interrupt it entirely. This gives you a place to insert human-in-the-loop checkpoints without having to bake them into the agent definition.

From a threat-modeling perspective, a Session is the unit of blast radius. When something goes wrong — the agent is manipulated, the prompt is injected, a tool is misused — the consequences play out within a single Session's container, event log, and tool invocations. Properties of Sessions therefore matter a great deal: how long they run, how much state they accumulate, what credentials they have access to, who can read their event log, who can submit user events into them.

2.4 Events

Events are the fourth concept and, in a sense, the substrate that the other three sit on top of. An Event is a single entry in the append-only log that constitutes the conversation between your application and a running Session. Events come in several types: user turns (input from your application), model outputs (text or tool calls from Claude), tool results (from the runtime, after executing a tool the model called for), and status updates (lifecycle and operational signals).

The event log is the backbone of how everything in a Session communicates. The model reads from it on every turn (that is what context assembly means at this level). The runtime writes to it as it executes tools and reports status. Your application reads from it via Server-Sent Events as the Session progresses, and can also fetch it in full at any time.

The implications of "everything is an event" are worth pausing on. There is no separate channel for trusted instructions versus untrusted content. The model's system prompt, the user's goal, the result of a web fetch from a hostile site, and an output from a third-party MCP server all end up in the same log, in the same general format, available to the model's next turn. The model has no architectural way to know which of these come from sources you trust and which do not. This is a deep property of the platform — it is not a flaw of one component you can fix, it is a consequence of how the system is structured — and it is the reason the Section 7 analysis treats the event log as the central object in the threat model rather than as a peripheral one.

The flip side is that the event log is also the single richest source of audit data you have. Every decision the model made, every tool it called, every result it received, is in the log, durably stored, and retrievable. For an industry where regulators increasingly want to know what an AI system actually did, that is a significant property — provided you actually capture and review it.

3. How it actually works: the lifecycle of a session

With the four concepts in hand, we can now walk through the end-to-end flow of bringing an agent to life and getting work out of it. The official documentation describes this as five phases: create an agent, create an environment, start a session, send events and stream responses, and steer or interrupt. The phases are useful, but the more illuminating picture is what is happening at each one — what crosses which boundary, what state is being created, and where the LLM is and is not involved.

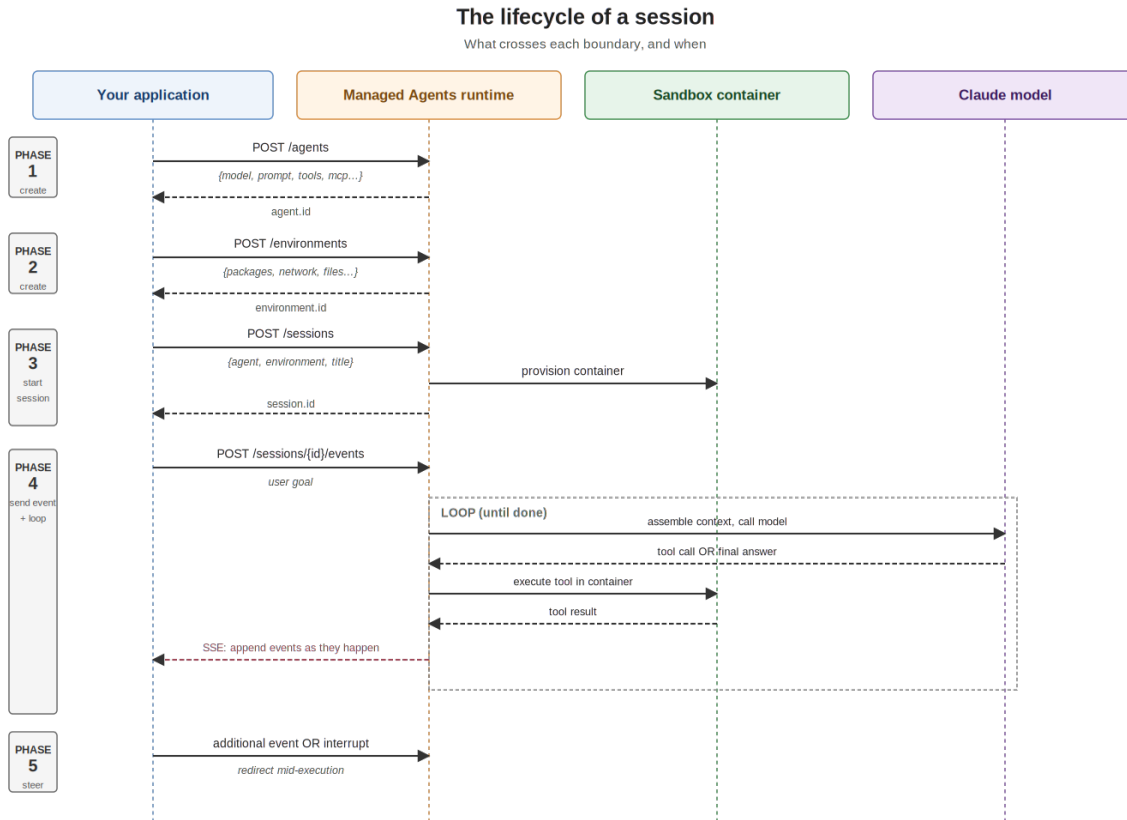


Figure 3.1 — The lifecycle of a session, shown as four lanes: your application, the Managed Agents runtime, the sandbox container, and the Claude model.

3.1 Phase 1 — Create an Agent

Your application makes one request to the Managed Agents API: "create an Agent with this model, this system prompt, these tools, these MCP servers, these skills." The runtime stores the definition and returns an Agent ID. Nothing is running yet. No container exists. The Claude model has not been called once. You have simply registered a configuration object with Anthropic's service and gotten back a handle to it.

Two things are worth noticing here. First, the Agent definition is durable: it is stored on Anthropic's side and can be referenced by ID indefinitely. This is what makes Agents reusable across many Sessions.

Second, the Agent definition is also reviewable: it is a finite, structured artifact you could subject to schema validation, policy checks, or formal review before it is created or each time it is updated. For a security-conscious deployment, that is the moment to insert your guardrails on tool selection, MCP server choice, and system prompt content.

3.2 Phase 2 — Create an Environment

Same shape, different content. Your application makes a request describing the container template: base image, packages, network rules, mounted files, environment variables. The runtime stores the template and returns an Environment ID. As before, nothing is actually running; you have registered a second configuration object.

In practice, you may have several Environments per Agent, or several Agents that share an Environment, or a one-to-one relationship. The split between the two definitions is deliberate: it lets you change the agent's mind (system prompt, tools) without rebuilding its body (container, packages), and vice versa. From a security architecture perspective, it also lets you separate concerns: an application engineer might own the Agent definition while a platform team owns the Environment, with each subject to its own review process.

3.3 Phase 3 — Start a Session

This is where things start running. Your application asks the runtime to start a Session, passing in an Agent ID, an Environment ID, and (typically) a title for the session. The runtime now does real work: it provisions a container according to the Environment template, attaches it to the Session, allocates an event log, and returns a Session ID.

The Claude model still has not been invoked. The Session is alive but idle, waiting for its first user event. The container exists; its filesystem is mounted; its network rules are in place. From here on out, Session-scoped costs (the per-session-hour runtime fee, in addition to standard token costs) start being incurred for the time the agent is actively working, though idle time — time the Session is waiting for input or for a tool — is not charged at the runtime rate.

3.4 Phase 4 — Send events and stream responses

This is the heart of the lifecycle and where the agent loop you read about in the conceptual material actually runs — except that, in Managed Agents, the loop runs inside Anthropic's runtime rather than inside your code.

Your application posts a user event into the Session: "Summarize today's CVEs that affect our stack." The runtime appends it to the event log and starts the loop:

- Assemble the context: system prompt, tool schemas, the event log so far, any retrieved skills.
- Call the Claude model with that context.

- Read back what the model emits: either a final answer or one or more tool calls.
- If a tool call: validate it, execute the tool inside the container, capture the result, append the result to the event log.
- If a final answer: append it to the event log and (when there is nothing else pending) treat the turn as complete.
- Stream every event back to your application as it happens, via Server-Sent Events.
- Repeat until the model judges the work complete or some other termination condition is met.

Several things deserve emphasis. The model is invoked once per loop iteration, not once per Session; a single user goal might result in dozens or hundreds of model calls as the agent works. The container persists across these iterations — file writes from one tool call are visible to the next. The event log is the single source of truth that everything reads from and writes to. And your application can disconnect at any point and reconnect later: the Session keeps running, and the event stream is durable.

3.5 Phase 5 — Steer or interrupt

The fifth phase is not really a separate phase — it is something that can happen at any time during phase 4. Your application can post additional user events into the event stream while the Session is running, and the runtime will fold them into the next context the model sees. This gives you a way to redirect the agent ("actually, focus only on CVEs with CVSS ≥ 7 "), to answer a clarifying question the agent asked, or to inject corrections.

You can also interrupt outright — stopping the current loop iteration and forcing the agent to reconsider. From a control-flow perspective, this is the place where human-in-the-loop oversight has its strongest leverage. The agent is autonomous within the Session, but the Session is not autonomous from you; the steerability of the platform is what makes that distinction possible.

3.6 Tying it back to the conceptual model

The lifecycle above is what the platform looks like. The lifecycle as it relates to the underlying agent architecture is the same loop you encountered in the conceptual material: an LLM at the centre, a runtime that mediates every interaction, tools that bridge the model's context and the outside world, memory layered through the context window and the persistent filesystem, and a sandbox that contains the blast radius. What Managed Agents has done is package that architecture as a service. The pieces are still there — they have not gone away — but most of them are now Anthropic's responsibility to operate, leaving you to focus on the parts where your domain knowledge actually matters: the agent's purpose, its capability surface, and the policy you want to enforce around it.

That packaging is a real productivity win, and it is also a real shift in your threat model. The next sections move from the architectural picture into the practical: what the interface actually looks like, how to get an agent running end-to-end, and where the new threat model leads.

4. A walkthrough of the interface

A natural question after the conceptual material is: where do you actually go to do this? In a product like a SaaS platform, the answer would be a URL and a login screen. In Managed Agents, the honest answer is: there is no single place. The product is API-first and surfaces itself through three different interfaces, each oriented toward a different kind of work. Understanding what each one is for — and what it isn't — saves you from going looking for capabilities in the wrong surface.

The three surfaces are the command line tool (the ant CLI), the SDKs (Python, TypeScript, and several others), and the Claude Console. The relationship between them is not hierarchical; they are three different ways into the same set of API endpoints, each with its own ergonomics and its own typical user.

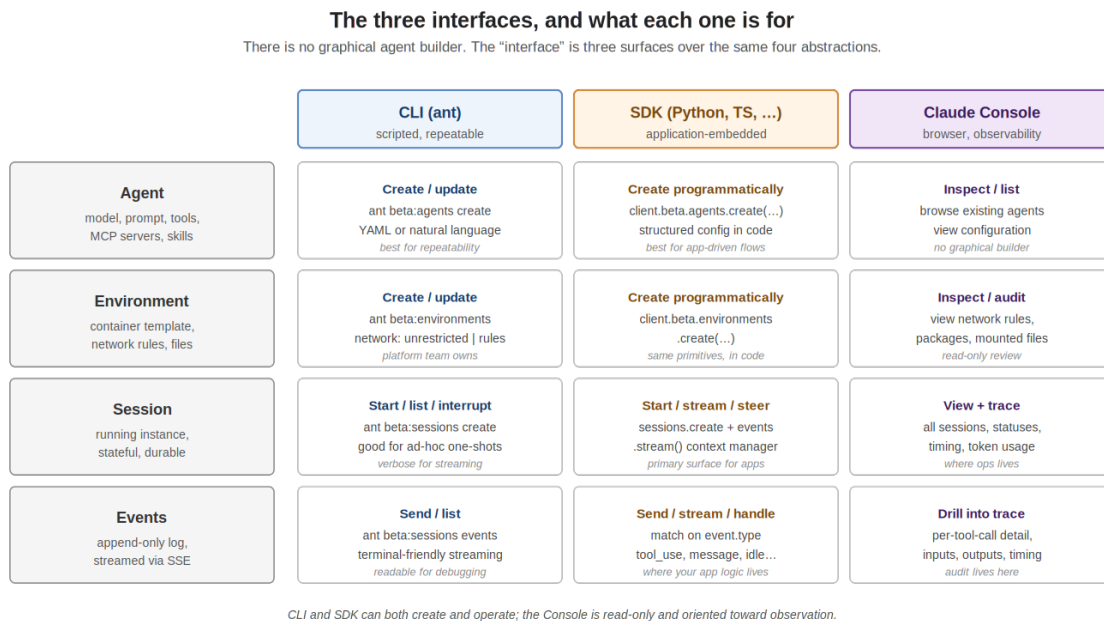


Figure 4.1 — The three interfaces (CLI, SDK, Console) mapped against the four core concepts. CLI and SDK can both create and operate; the Console is read-only and oriented toward observation.

4.1 The ant CLI

The CLI is the simplest way to interact with Managed Agents. It is a single binary, **ant**, that exposes the API surface as a set of subcommands (**ant beta:agents create**, **ant beta:sessions create**, and so on). Output is text; arguments are flags or short JSON snippets; the streaming session view prints events as they arrive.

The CLI is well suited to several specific kinds of work. It is excellent for one-off ad hoc operations — spinning up a session to test a prompt change, listing recent sessions, fetching the events from a session you want to inspect. It is also the right surface for shell scripts that wrap the lifecycle: a cron job that

runs your agent every morning, a CI step that smoke-tests an agent definition, an operations playbook that triggers an agent in response to a paging signal.

Where the CLI is less well suited is anything that involves rich application logic. Once you need to react to specific event types in non-trivial ways — approving a tool call here, denying one there, persisting structured output to a database, calling out to your own services — the CLI starts to feel like the wrong shape and the SDK is the better choice. The CLI is for orchestration; the SDK is for application code.

A small but useful detail: agent definitions can be passed to the CLI as YAML files or as natural-language descriptions. The natural-language form lets you describe the agent in prose and have Anthropic’s tooling produce the structured definition. For learning, that lowers the friction. For production, the YAML form is what you want, because it is reviewable, diffable, and version-controllable. Treat the natural-language form as a sketchpad and the YAML form as the source of truth.

4.2 The SDK

The SDK is the surface where most application-level work happens. It is available in several languages — Python, TypeScript, Go, Java, C#, Ruby, and PHP — and provides typed bindings for every Managed Agents endpoint. For this tutorial, we will use the Python SDK; the conceptual structure is identical across languages and most of what is said here translates directly.

Two things distinguish the SDK from the CLI. The first is structure: events are typed objects with attributes you can inspect and dispatch on, rather than text you have to parse. The second is integration: the SDK runs inside your application, which means it has access to your application’s state, can call your application’s functions, and can hand its output to your application’s downstream pipeline.

The SDK’s primary surface is the four-method pattern: create an agent, create an environment, create a session, and stream events. The streaming method is a context manager (an async iterator in TypeScript) that yields events as they arrive. The shape of any application using Managed Agents is, at its heart, a loop over that event stream that dispatches on event type — message, tool use, status idle, requires confirmation — and does the right thing for each.

We will see the actual code in Section 6. For now, the point is that the SDK is where your application’s logic meets the agent’s execution, and most of the engineering work in adopting Managed Agents is on this surface.

4.3 The Claude Console

The Console is the third surface and it is markedly different from the other two. It is a web application at platform.claude.com, accessed through a browser, oriented entirely around observation rather than creation. You do not, today, build agents or define environments in the Console; you build them in code (CLI or SDK) and you observe them in the Console.

What the Console gives you is a browsable view of your sessions: a list, filterable by status, with each session expandable into a full event-by-event trace. For any given event you can drill into the inputs, the outputs, the timing, and the token usage. For any given session you can see the cumulative usage statistics, the agent and environment that were used, and the lifecycle (when it started, when it went idle, what stopped it). For audit and debugging this is genuinely useful — especially for sessions that ran a long time ago or that ran on a schedule when nobody was watching.

The mock in Figure 4.2 is a stylized representation of what the session trace view looks like. It is not a literal screenshot — the Console will continue to evolve through the beta period — but the structural elements (sessions list, session metadata bar, event timeline, event-detail inspector) are what you will see.

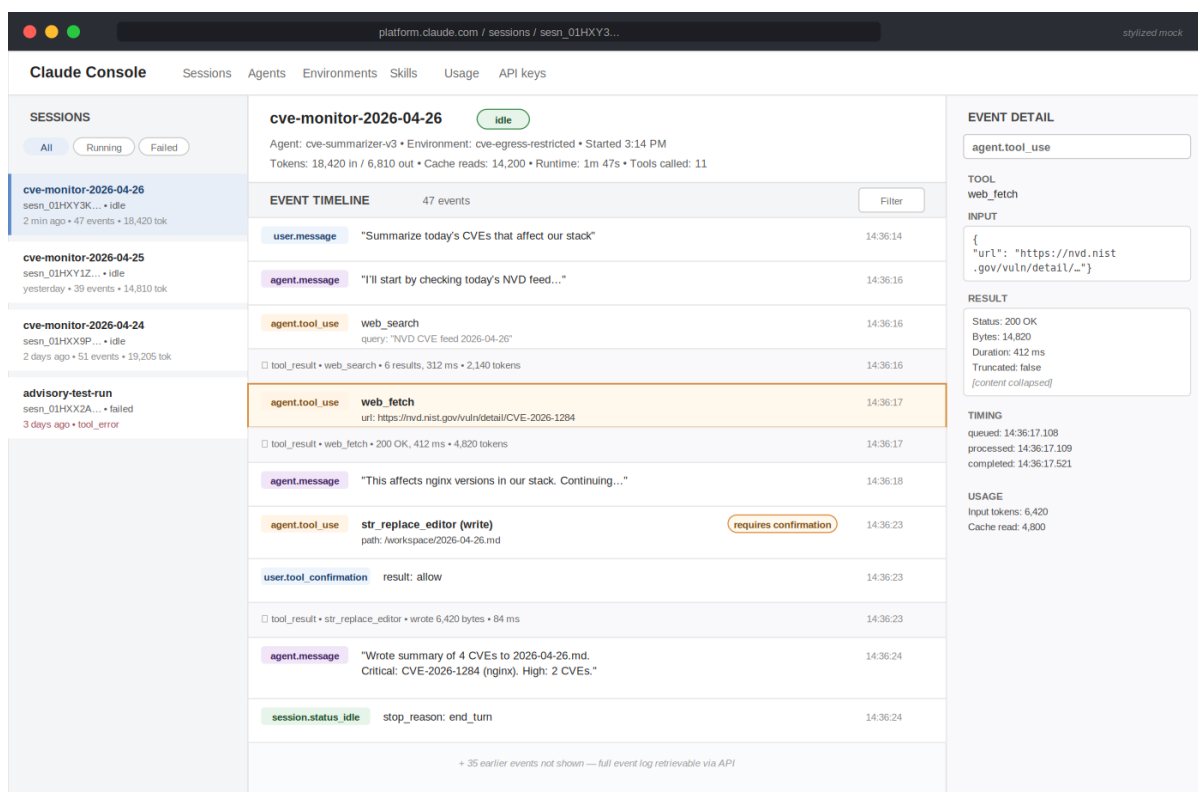


Figure 4.2 — A stylized mock of the Claude Console session trace view. Sessions list on the left, event timeline in the centre, per-event detail on the right. (Mock, not a screenshot.)

What the Console gives you

- **Session inventory.** A live list of every session your organization has run, filterable by status (running, idle, failed) and searchable by title or agent.
- **Event timeline.** Every event in the session, in order, color-coded by type. User messages, agent messages, tool calls, tool results, status events — all visible in one chronological feed.
- **Event detail inspector.** Click any event to see its full content: the exact arguments to a tool call, the full text of the result, timing breakdowns, token counts.

- **Cumulative metrics.** Token usage (input, output, cache reads, cache writes), runtime, number of tool calls, all rolled up at the session level.
- **Session metadata.** Which Agent and Environment definitions a session referenced, when it started, when (and how) it stopped.

What the Console does not give you

- **A graphical agent builder.** There is no point-and-click form for defining an Agent or an Environment. Definitions are code or YAML.
- **Real-time control.** You cannot send user events into a running session from the Console UI; that capability lives in the CLI and SDK. The Console is, today, primarily a read surface.
- **Cross-session analytics.** You can see one session at a time in detail, or browse the list, but the Console is not (yet) a dashboard for "how are all my agents doing this week." For that, you fetch the data via the API and graph it yourself.
- **Alerting.** There is no built-in alerting for failed sessions, runaway tool use, or budget overruns. Alerting is your responsibility, and it lives in your code or your existing observability stack.

It is worth saying that the Console is the youngest of the three surfaces and is likely to gain capabilities over the beta period. The shape sketched above reflects the product as of April 2026. If you are reading this materially later than that, expect the Console to have grown.

4.4 Choosing between them

A simple heuristic: use the CLI for shell-shaped work (one-off operations, scheduled jobs, scripts that wrap the lifecycle), use the SDK for application-shaped work (anything that needs to react to events with structured logic), and use the Console for observation-shaped work (audit, debugging, "what did this thing actually do"). Most non-trivial deployments end up using all three. The CVE monitor we will build in Section 6 is structured exactly that way: a YAML agent definition deployed through the CLI, a Python invocation script using the SDK, and the Console as the place we go when we need to see what happened on a particular run.

5. Getting started: prerequisites and first agent

This section is the literal getting-started checklist: what you need before you start, how to install the tooling, and a minimal working example to confirm everything is in place. We will keep it deliberately small. The point of this section is not to teach you how to build a useful agent — that is Section 6 — but to walk through the mechanics of getting an agent running at all.

5.1 Prerequisites

To use Managed Agents you need three things:

- **A Claude Console account.** Created at platform.claude.com. Free to set up; usage is billed against the API. If you already use the Anthropic API in your organization, you have one of these.
- **An API key.** Generated from platform.claude.com/settings/keys. Treat it as a credential of equivalent sensitivity to a long-lived cloud-provider key: never commit it, never put it in URLs, never paste it into prompts. Set it as an environment variable: `export ANTHROPIC_API_KEY="..."`.
- **The beta header.** All Managed Agents requests require `anthropic-beta: managed-agents-2026-04-01`. The SDK and CLI set this for you. If you are calling the API directly with curl, you will set it yourself.

Beyond those, the only optional prerequisite is access to the research-preview features (multi-agent, outcomes), which require a separate request. Those are not needed for anything in this tutorial.

On API key handling

API keys are bearer credentials with broad authority over your organization's API usage. The standard precautions apply, with one Managed-Agents-specific note: a leaked key allows an attacker not only to consume tokens, but to create and run sessions in your account, which incurs additional per-session-hour runtime costs and could be used to mount a low-and-slow exfiltration via your own MCP servers. Rotate aggressively, scope where possible, and monitor the Usage view in the Console for anomalies.

5.2 Installing the tools

Install the CLI. The recommended path on macOS is Homebrew; on Linux/WSL the installer is a curl-piped script. Verify with `ant --version`. There are also Go and other distributions for environments where neither Homebrew nor curl is appropriate.

Install the Python SDK:

```
pip install anthropic
```

Verify the SDK and key are working:

```

verify_setup.py
import anthropic

client = anthropic.Anthropic() # picks up ANTHROPIC_API_KEY from env
# A trivial Messages API call to confirm credentials work:
resp = client.messages.create(
    model="claude-opus-4-7",
    max_tokens=64,
    messages=[{"role": "user", "content": "Say OK if you can hear me."}],
)
print(resp.content[0].text)

```

If that prints something resembling "OK", your credentials and SDK are in place and you are ready to use Managed Agents. The Messages API call above does not exercise Managed Agents specifically, but a working Messages call is a precondition; if it fails, nothing in the rest of this tutorial will work either.

5.3 A minimal first agent

We will build the smallest possible Managed Agents example: an agent whose only job is to fetch a URL and produce a one-paragraph summary. This exercises the full lifecycle (create agent, create environment, start session, send event, stream events, observe completion) without introducing any of the complexity we will need in Section 6. Everything below uses the CLI for the durable definitions and the Python SDK for the running of the session, mirroring the recommended split.

Step 1 — Create the agent

```

ant beta:agents create \
  --name "url-summarizer-minimal" \
  --model '{id: claude-opus-4-7}' \
  --system "You are a concise summarizer. Given a URL, fetch it and produce a single
paragraph (4-6 sentences) capturing the main points. Cite the source URL at the end. Do
nothing else." \
  --tool '{type: agent_toolset_20260401}'

```

The CLI prints the created agent's ID. Save it; you will need it in step 3. `agent_toolset_20260401` is shorthand for "give the agent the standard set of pre-built tools" — bash, file operations, web search and fetch. For a real production agent you would narrow this down; for our minimal example, the bundle is fine.

Step 2 — Create the environment

```

ant beta:environments create \
  --name "minimal-env" \
  --config '{type: cloud, networking: {type: unrestricted}}'

```

Save the environment ID too. The `unrestricted` network setting means the container can reach anywhere on the internet — not what you want for production, but the simplest thing for a first run. We will tighten this in Section 6.

Step 3 — Start a session and stream events

```

first_agent.py
import anthropic

client = anthropic.Anthropic()

AGENT_ID = "agt_..."      # from step 1
ENVIRONMENT_ID = "env_..." # from step 2

session = client.beta.sessions.create(
    agent=AGENT_ID,
    environment_id=ENVIRONMENT_ID,
    title="minimal first run",
)
print(f"Session: {session.id}")

with client.beta.sessions.events.stream(session.id) as stream:
    client.beta.sessions.events.send(
        session.id,
        events=[{
            "type": "user.message",
            "content": [{
                "type": "text",
                "text": "Summarize https://www.anthropic.com/research/building-effective-
agents",
            }],
        }],
    )

    for event in stream:
        match event.type:
            case "agent.message":
                for block in event.content:
                    if hasattr(block, "text"):
                        print(block.text, end="", flush=True)
            case "agent.tool_use":
                print(f"\n[tool: {event.name}]", flush=True)
            case "session.status_idle":
                print("\n\n[done]")
                break

```

Run that script. You should see the agent narrate what it is doing, call the `web_fetch` tool, then produce a paragraph summarizing the page. The whole thing takes a few seconds and a few thousand tokens.

Step 4 — Look at it in the Console

Open platform.claude.com, navigate to Sessions, and find the session you just ran. Open it. You should see the same trace structure as Figure 4.2: the user message, the agent’s reasoning, the tool call, the tool result, the final agent message, and the `status_idle` event that marked the end. Click into the `web_fetch` event to see its inputs and outputs in detail.

That is the loop you will use over and over again: define an agent in code, run a session in code, observe in the Console. Every example for the rest of this tutorial follows that same pattern. The next section uses the same primitives to build something useful.

6. Building and deploying a CVE monitor end-to-end

We will now build a real, useful agent. The scenario is one you would actually encounter as a security architect: a daily monitor that reads the day's newly published CVE advisories, cross-references them against your organization's technology stack, and produces a structured summary of the ones that matter. The agent runs in a tight network sandbox, requires confirmation before writing files, and is wrapped for repeated invocation under a scheduler.

The example deliberately exercises several capabilities at once: web search and fetch, the bash tool, file operations, network egress restriction, tool-confirmation policies, and the SDK's event-handling pattern. By the end, you will have seen most of what Managed Agents offers in a working context, and you will have a template you could adapt to other monitoring use cases (security advisories, vendor announcements, regulatory updates) with relatively small changes.

6.1 The agent definition

We will define the agent as YAML rather than CLI flags, because the definition is something we will want to review, version, and treat as code. Save the following as `cve_summarizer.yaml`:

```
cve_summarizer.yaml
name: cve-summarizer-v3
model:
  id: claude-opus-4-7
system: |
  You are a security analyst. Each day, you produce a brief summary of newly
  published CVE advisories that affect the organization's technology stack.

  The stack inventory is at /workspace/stack.json. Read it before searching;
  do not assume what is in the stack. The inventory contains canonical product
  names and version ranges.

  Workflow:
  1. Identify CVEs published in the last 24 hours from the NVD feed.
  2. For each candidate CVE that mentions a product in the stack, fetch the
     full advisory and confirm whether the affected version range overlaps
     with the version we run.
  3. Discard CVEs that do not affect us. For the rest, capture: CVE ID,
     affected product, severity (CVSS v3.1), one-paragraph impact summary,
     and a one-line mitigation note.
  4. Write the result as Markdown to /workspace/out/<YYYY-MM-DD>.md.
  5. End your turn with a one-paragraph summary message that names the
     output file and the count of critical/high/medium findings.

  Be terse. Do not speculate. If a CVE's applicability is genuinely
  uncertain, include it under a separate "needs review" heading and explain
  what would resolve the uncertainty.

tools:
```



```
- type: agent_toolset_20260401
  permission_policy:
    file_write: requires_confirmation
    bash: requires_confirmation
```

A few choices in this definition are worth dwelling on, because they are the load-bearing security decisions and they are the kinds of choices you will be reviewing on every agent you deploy.

Stack inventory as a mounted file. We give the agent the stack inventory as a file rather than embedding it in the system prompt. This means the inventory can be updated without rebuilding the Agent definition, and it means a long stack list does not bloat every model call. It also means the inventory enters the model’s context only when the agent reads it on a given run, which is mildly cleaner from a token-economics perspective.

Tool confirmation on writes and bash. The `requires_confirmation` policy on `file_write` and `bash` means that whenever the model wants to write a file or run a shell command, the runtime pauses the session and emits a `requires_action` status. Our application code then has to decide whether to allow or deny each of those calls before the agent can proceed. This is the platform’s built-in human-in-the-loop primitive, and it is the right hook for the kinds of actions whose blast radius warrants supervision.

Reads are not gated. Web search, web fetch, and file reads are not subject to confirmation. This is a deliberate trade-off: gating reads would make the agent untenably noisy and would not buy us much, because the blast radius of a read in this Environment is limited by the network egress policy (next subsection) rather than by who called it.

Create the agent:

```
ant beta:agents create --from cve_summarizer.yaml
```

6.2 The environment definition

The Environment is where we exercise the most leverage. We will configure the container to mount our stack file, install nothing extraneous, and — most importantly — restrict network egress to only the hosts we actually expect the agent to talk to. Save as `cve_env.yaml`:

```
cve_env.yaml
name: cve-egress-restricted
config:
  type: cloud
  networking:
    type: restricted
    allow:
      - host: nvd.nist.gov
      - host: services.nvd.nist.gov
      - host: api.first.org           # for CVSS lookups
      - host: www.cisa.gov           # for KEV catalog
```

```

- host: cve.mitre.org
- host: github.com          # for vendor advisories
- host: security-advisories.org
packages:
- python3
- jq
- curl
files:
- mount: /workspace/stack.json
  source: ./stack.json
- mount: /workspace/out/.keep
  source: ./empty

```

The egress allow-list is the security primitive. No matter what the agent decides to fetch, no matter how cleverly an attacker manipulates a tool result to suggest the agent retrieve content from a malicious host, the container cannot reach hosts outside the allow-list. This is enforced by the runtime; the model does not get a vote. If you take one design decision away from this section, take this one: **restrict the Environment's network surface to the smallest set of hosts the agent actually needs.**

Create the environment:

```
ant beta:environments create --from cve_env.yaml
```

6.3 What the agent does, end to end

Before we look at the invocation code, it is worth seeing what actually happens when the agent runs. Figure 6.1 shows the sequence of tool calls in a typical run. The important property to notice is that no human wrote this sequence. We did not tell the agent "first search, then fetch, then read the stack file, then synthesize, then write." We described the goal in the system prompt and gave it tools; the model composed the sequence at runtime. Tomorrow's run might compose a different sequence — perhaps the agent reads the stack file first, perhaps it fetches advisories from CISA before NVD, perhaps it abandons a partially-filled summary and starts again because something in the stack file changed. That is the dynamism of an agent.

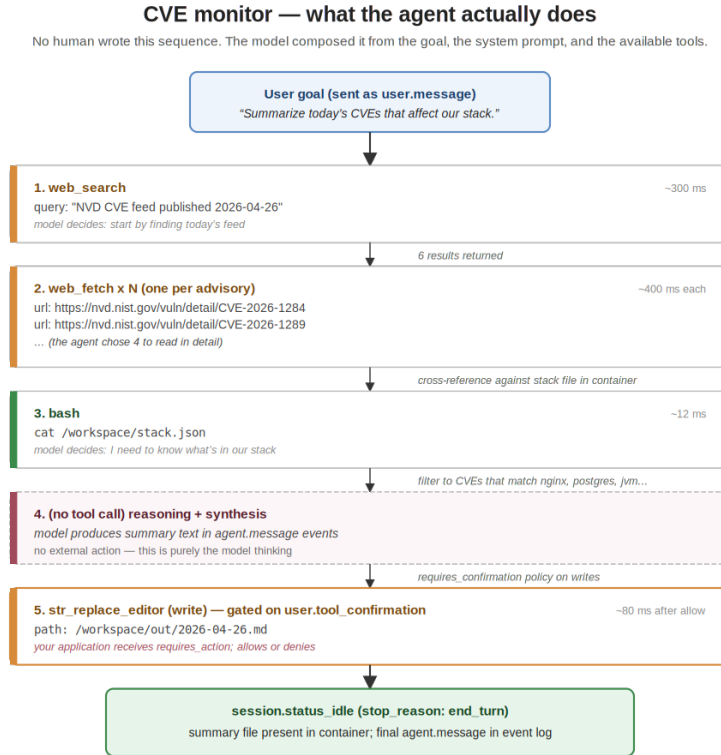


Figure 6.1 — The CVE monitor's tool-call sequence. The agent decides each step at runtime based on the goal, the system prompt, and the available tools.

6.4 The invocation script

Here is the Python script that drives a single run. We will walk through it in pieces. The full script is around 80 lines; an abridged version follows.

```

cve_run.py (abridged)
# cve_run.py
import datetime as dt
import json
import os
import sys

import anthropic

AGENT_ID = os.environ["CVE_AGENT_ID"]
ENVIRONMENT_ID = os.environ["CVE_ENV_ID"]

client = anthropic.Anthropic()

today = dt.date.today().isoformat()
prompt = (
    f"It is {today}. Summarize today's newly published CVEs that affect "
    "our stack as defined in /workspace/stack.json. Follow your standard workflow."

```

```

)

session = client.beta.sessions.create(
    agent=AGENT_ID,
    environment_id=ENVIRONMENT_ID,
    title=f"cve-monitor-{today}",
)
print(f"[session] {session.id}", file=sys.stderr)

allowed_writes = {f"/workspace/out/{today}.md"}    # policy: only today's file
denied_count = 0
final_message = ""

with client.beta.sessions.events.stream(session.id) as stream:
    client.beta.sessions.events.send(
        session.id,
        events=[{"type": "user.message",
                  "content": [{"type": "text", "text": prompt}]}],
    )

    for event in stream:
        match event.type:

            case "agent.message":
                for block in event.content:
                    if hasattr(block, "text"):
                        final_message = block.text    # last one wins
                        print(block.text, end="", flush=True)

            case "agent.tool_use":
                print(f"\n[tool: {event.name}]", file=sys.stderr)

            case "session.status_idle":
                stop = getattr(event, "stop_reason", None)
                if stop and stop.type == "requires_action":
                    handle_pending_actions(client, session.id, stop, allowed_writes)
                else:
                    break

def handle_pending_actions(client, session_id, stop, allowed_writes):
    """Approve or deny tool calls awaiting confirmation."""
    confirmations = []
    for event_id in stop.event_ids:
        # In practice you'd fetch the event by ID and inspect it.
        pending = client.beta.sessions.events.retrieve(session_id, event_id)
        decision = decide(pending, allowed_writes)    # your policy
        confirmations.append({
            "type": "user.tool_confirmation",
            "tool_use_id": event_id,
            "result": decision,
        })
    client.beta.sessions.events.send(session_id, events=confirmations)

```

A few notes on the structure of this script.

The `handle_pending_actions` helper is where your application-level policy lives. When the agent emits a tool call that the Agent's policy has flagged as `requires_confirmation`, the runtime pauses the session and your script gets a `session.status_idle` event with `stop_reason: requires_action` and a list of event IDs awaiting confirmation. You inspect each one and emit either `result: "allow"` or `result: "deny"` for it. In the example above the policy is simple: allow writes to today's output path, deny everything else. A real deployment would probably also examine the proposed bash commands, log every decision, and alert on denials.

The dispatch is on event type. The `match` statement on `event.type` is the canonical SDK pattern. Every event you care about has a string type (`agent.message`, `agent.tool_use`, `session.status_idle`, `user.tool_confirmation`, and so on); your application logic is essentially a routing layer over those types.

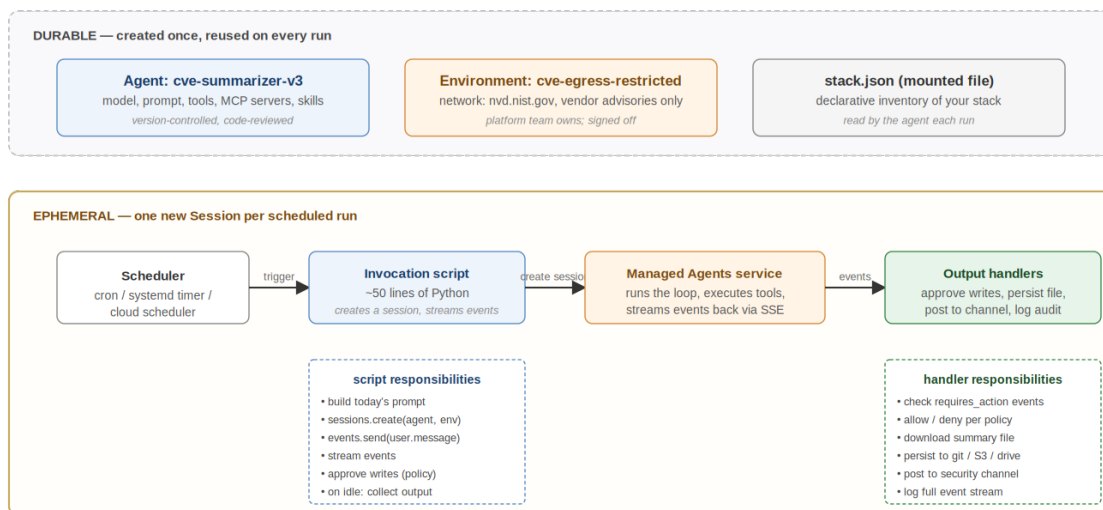
The script does not contain any retry logic. In production you would wrap the stream in code that handles network disconnections by reconnecting and resuming the stream from where you left off (the durable event log makes this straightforward; the SDK provides a `cursor` parameter for it). For clarity, that is omitted from the example above.

6.5 Deploying for repeated invocation

"Deployment" of this agent mostly means parameterizing the script we just wrote and putting a scheduler in front of it. The Agent and Environment definitions are durable: created once, referenced by ID forever. The Session is created fresh on each run. The script is the small piece of glue that ties them together for a particular invocation.

Deployment shape: from one-shot to repeated daily run

"Deployment" mostly means parameterizing the invocation. The Agent and Environment definitions stay still.



The agent is autonomous within the session; the session is not autonomous from the surrounding pipeline.

Figure 6.2 — Deployment shape. Durable definitions on top; ephemeral per-invocation work on the bottom. The script is small; the agent and environment do most of the work.

Concretely, a daily deployment looks like this:

- A scheduler (cron, systemd timer, a cloud scheduler, an existing security-orchestration platform) fires once a day at a chosen time.
- The scheduler invokes `cve_run.py` with the agent and environment IDs in environment variables.
- The script creates a fresh Session, posts the day's prompt, streams events, approves the single expected write, and waits for the session to go idle.
- On idle, the script downloads the produced summary file from the container, persists it to your archive (a git repo, an S3 bucket, a wiki, your security team's drive), and posts a one-paragraph notification to your security channel with the link.
- The full event log remains in Anthropic's storage and is browsable in the Console for as long as your retention policy permits.

A useful property of this shape is that the durable definitions are the "code" — the version-controlled, code-reviewed, change-managed part — and the per-run work is reduced to a tight loop over events. Updating the agent's behavior is a YAML change, code-reviewed and re-deployed with one CLI command. Updating the network policy is a YAML change to the Environment. The script almost never changes.

A point worth dwelling on

Notice how much of the security posture lives outside the model. The system prompt sets behavior; the policy on tool use sets confirmation; the network allow-list sets reachability; the script's policy sets which writes are accepted; the schedule sets cadence. The model's autonomy operates within an envelope that is shaped by all of those layers, none of which the model gets to choose. This is the right way to deploy agents in security-relevant contexts: do not rely on the prompt to enforce policy. Use it to express intent, and rely on the layers around it to enforce constraints.

6.6 What you have built, and what comes next

In about 100 lines of YAML and Python, you have a security-domain agent that runs autonomously, talks to a constrained network surface, requires human approval for the actions whose blast radius warrants it, produces a structured daily artifact, and is observable end-to-end in the Console. The pattern generalizes: the same shape (durable agent + restricted environment + scheduler-driven invocation script + tool-confirmation policy + observable outputs) applies to a vendor-advisory monitor, a regulatory-update tracker, a configuration-drift detector reading from snapshots, or any number of similar tasks.

What you have not yet seen is what could go wrong with all of this. The CVE monitor as built is reasonable but not bulletproof; an attacker who could influence what the agent retrieves from a vendor advisory page might be able to influence what it writes to the output file. The egress allow-list is good but the tool result is still a vector. The tool-confirmation policy is good but only as good as the script's decision logic. Section 7 walks through these threat-model questions methodically; Section 8 closes the tutorial with a summary of where Managed Agents fits in the security architect's toolbox.

7. The security architect's lens

Everything up to this point has been preparation for this section. We have built a working agent, deployed it under a scheduler, and made several deliberate-looking security choices along the way. Now we step back and ask the questions a security architect is paid to ask: what are we actually trusting here, what are we no longer in control of, what new attack surfaces does this architecture create, and what should we be doing about all of that?

I want to be careful about how strong the claims in this section are. Managed Agents is a young product. Some of what follows is grounded in well-understood properties of any sandboxed-LLM-with-tools architecture; some of it is speculation about how attackers will adapt. Where I am confident, I will be plain. Where I am extrapolating, I will say so. The goal is not to scare you off using the platform — it is genuinely useful, and many of the new attack surfaces are no worse than the corresponding surfaces in a do-it-yourself agent. The goal is to make sure your threat model has the right shape going in, so that when you do adopt Managed Agents you do it deliberately rather than by accident.

7.1 What moved to Anthropic's trust boundary

Adopting Managed Agents is, before anything else, an act of trust placement. There is a long list of things that, if you build an agent on the Messages API, you have direct custody of. Most of those move to Anthropic when you adopt Managed Agents. It is worth saying out loud what they are.

The most important is the agent loop itself. The runtime that decides which event to send to the model next, that parses the model's output, that dispatches tool calls, that retries on transient failures, that compacts context when it grows too large — all of that is now Anthropic's code, running on Anthropic's servers. The integrity of that loop is the integrity of your agent. If the loop has a bug, your agent has a bug. If the loop has an exploitable defect, your agent is exploitable.

Closely related is the sandbox. The container in which tools run, the kernel-level isolation that keeps one customer's session away from another's, the network policy enforcement that backs the Environment's allow-list, the resource limits that prevent runaway tool calls from consuming arbitrary compute. All of this is operated by Anthropic, audited by Anthropic, and patched by Anthropic. You have visibility into the policy you defined; you do not have visibility into the implementation that enforces it.

Then there is session storage. The event log — every user message, every model output, every tool call, every tool result, every status event for every session you have ever run — lives in a database operated by Anthropic. The retention is set by Anthropic. The encryption-at-rest is by Anthropic. The access controls within Anthropic's infrastructure are by Anthropic. If your agent processed sensitive material, that sensitive material is in their database. The standard "what happens if Anthropic is breached" question applies, with the same severity you would attach to any other SaaS provider holding similar data.

Finally, the operational substrate: prompt caching, compaction, checkpointing across network disconnections, the rate-limiting and queueing infrastructure, the metrics and tracing pipeline that feeds the Console. None of this is your code. None of it is testable end-to-end from your side. You learn about its behavior through observed outputs and through whatever Anthropic chooses to expose in their documentation.

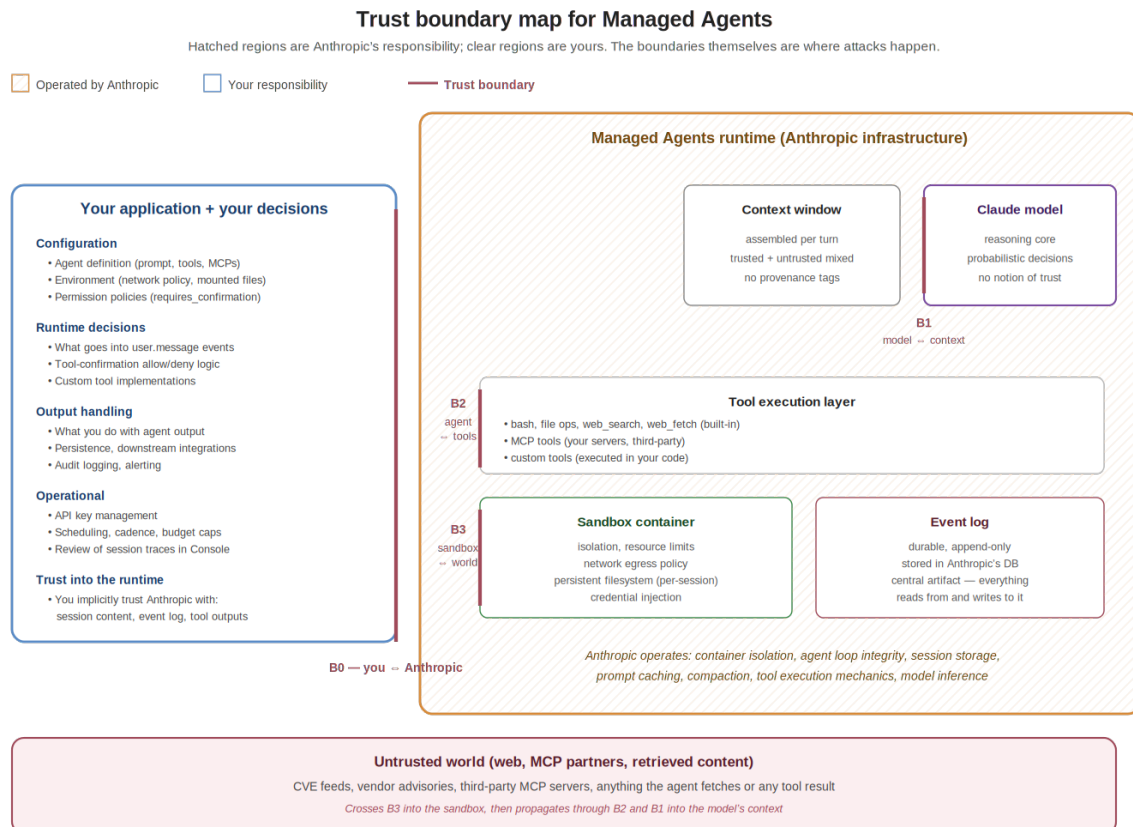


Figure 7.1 — Trust boundary map for Managed Agents. Hatched regions are operated by Anthropic; clear regions are your responsibility. Four boundaries (B0–B3) define where attacks and controls live.

Figure 7.1 redraws the trust-boundary picture from earlier in the conceptual material with these ownership lines made explicit. The thing to notice is that the boundary B0 — between you and Anthropic — is new. It did not exist when the agent loop ran in your own infrastructure. Adopting Managed Agents is the choice to draw that boundary at all, and the choice to put almost everything except your application code on the far side of it.

None of this is unique to Managed Agents. Adopting any SaaS platform involves the same kind of trust placement. What is worth being deliberate about is that, with agentic AI specifically, the things you are placing on the far side of the boundary are unusually consequential: the loop that makes decisions on your behalf, the sandbox that constrains what those decisions can do, and the log of everything those decisions touched. A SaaS spreadsheet provider does not get to decide what your spreadsheet does. A managed agent runtime, by definition, does.

What this means in practice

Use the Console's session traces as part of your audit trail — not as the audit trail. Pull session events into your own logging infrastructure on completion, where they live under your retention policy and are searchable alongside the rest of your security telemetry.

Make the set of actions an agent can take small and explicit. The defense-in-depth principle still applies: even if Anthropic's sandbox is rock solid, an agent that can write to S3 and reach the public internet has a much larger blast radius than one that cannot.

Treat the Anthropic relationship the way you treat your other critical SaaS dependencies. Vendor security questionnaires, BCP/DR planning, contractual data handling, audit rights to the extent your agreements permit — the standard SaaS playbook applies.

7.2 What stayed firmly with you

The other side of that boundary is just as important. Several things did not move to Anthropic when you adopted Managed Agents, and these are precisely the places where your defensive leverage now lives. Naming them clearly is the second half of getting the threat model right.

The Agent definition

What model the agent uses, what its system prompt says, which tools it has, which MCP servers it can call, which skills it has loaded — all of this is your decision and lives in code or YAML on your side. The Agent definition is the place where most of your offensive-security relevant choices end up: the choice to give the agent a broad tool set or a narrow one, the choice to allow it to call a third-party MCP server or to keep it hermetic, the choice of which model (and therefore what capability surface) to use. Treat the Agent definition as security-critical configuration. Review it like you would review IAM policies or firewall rules.

The Environment definition

The container template, the package list, the network egress policy, the mounted files — also your decisions, also your code or YAML. We saw in Section 6 that the Environment's network policy is the most load-bearing single security control in the whole system: it is where "the agent can only fetch from these hosts" is enforced, and that enforcement is independent of anything the model does or says. The same logic applies to mounted files (do not mount what does not need to be there) and to packages (do not install what does not need to be installable).

What enters the event stream from your side

Every `user.message` event you send is content you have put into the agent's context. Every `user.tool_confirmation` decision is a security-relevant act. Every custom tool result you return is content you have introduced. None of this comes from Anthropic; all of it comes from your application. The

hygiene of your inputs is your responsibility — if your application takes user input from the public internet and pipes it into the `user.message` of a privileged agent, that is your prompt-injection problem to solve, not Anthropic's.

Tool-confirmation policy logic

When the agent emits a tool call gated by `requires_confirmation`, the runtime pauses and asks your application to decide. The decision is yours. The platform is providing the hook; you are providing the policy. A simple policy ("only allow writes to today's output path") is what we used in Section 6. A more sophisticated deployment might inspect the proposed bash command, run it through a static-analysis check, log the decision to a SIEM, alert on certain patterns. The point is that the gate exists; what runs at the gate is up to you.

Output handling and downstream integration

The agent produces an event stream and (in our case) a file in the container. What happens next — where the file goes, who gets notified, how the events are persisted, what alerts fire — is entirely your code. The agent's output is not gospel; it is content that flows into your downstream pipeline and should be subject to the same validation and integrity checks you would apply to content from any other source. If the agent produces a Markdown summary, the summary should not, for example, be rendered into your security wiki without sanitization, because an attacker who manipulated the agent's reasoning could influence what is in that summary.

7.3 The new attack surfaces

With ownership clear, we can now look at where attacks actually happen. There are several attack surfaces specific to agentic systems, and several that take new forms when the system is hosted by a managed platform. I will walk through them in roughly the order you would encounter them in a threat model.

Prompt injection through tool results

This is the canonical agentic-AI security problem and it does not go away on Managed Agents. When the agent fetches a web page, the contents of that page enter the event stream as a `tool_result` event. When the next model turn assembles its context, that `tool_result` is in there alongside the system prompt, the user goal, and the agent's prior reasoning. The model has no architectural way to distinguish "instructions from the system prompt that I should follow" from "instructions that were embedded in a hostile web page that I should ignore." Both are text in the same context window.

In our CVE monitor, this means: an attacker who can plant content on a host the agent fetches — say, by getting a malicious advisory published on a vendor blog the agent might read — can attempt to influence what the agent does next. The egress allow-list narrows the set of hosts an attacker would need to compromise (NVD, CISA, vendor advisories), but it does not eliminate the surface. Defending

the model's reasoning against adversarial content in its own context is an active research area; there is no clean solution today. The mitigations available are mostly architectural: limit what the agent can do (so the consequences of a successful injection are bounded), gate sensitive actions (so a successful injection cannot complete an exfiltration without triggering a confirmation), and audit trail everything (so you can detect after the fact).

Privilege escalation through chained actions

A subtler version of the same problem. Each individual tool call your agent makes is, on its face, fine: read a file, fetch a URL, write a summary. The composition can be harmful in ways no individual call is. An agent that reads sensitive content, then fetches an attacker-controlled URL with that content encoded in a query string, has just exfiltrated the content using two innocuous tool calls. None of the calls would fail an obvious policy check; the harm is in the chain.

Managed Agents has primitives that help here — the network egress policy bounds where data can flow, the tool-confirmation policy bounds which actions complete unattended — but it does not have a built-in notion of cross-action data flow analysis or taint tracking. If you want to reason about chains, you have to do it yourself, in your tool-confirmation logic and in your post-hoc audit of the event log. For a high-stakes deployment, building a small "what was read, what was written, what was sent where" tracker over the event stream is a worthwhile investment. The event log gives you all the raw material; turning it into actionable signal is on your side.

Memory poisoning across sessions

In our CVE monitor we did not use long-term memory beyond the mounted stack file, which is read-only from the agent's perspective. Many other agent designs do use persistent memory — vector stores for retrieval-augmented generation, episodic memory across sessions, learned-procedure stores. The Memory tool in Managed Agents is one such mechanism. Anywhere the agent writes to a store that is read on future runs, you have a write-once-exploit-many surface. An attacker who can influence one session's output has potentially influenced every future session that retrieves it.

The mitigations are conventional but worth restating. Treat any persistent store the agent can write to as content of unknown provenance on the next read. Apply the same input-hygiene controls you would apply to content from outside your trust boundary. Partition by tenant aggressively in multi-tenant deployments. And limit what the agent can write to memory in the first place — if a piece of state can be re-derived from authoritative sources on each run, prefer re-deriving it over storing it.

Supply chain risk in the tool ecosystem

Managed Agents lets you connect MCP servers (your own or third-party) and load skills. Both of these are code or instructions running inside your agent's context, often originating from outside your organization. The trust questions are the standard supply-chain questions, with one wrinkle specific to agentic systems: a malicious MCP server does not need to deliver an obvious payload. It just needs to

influence what the model decides to do. A tool description that says "use this tool whenever you encounter sensitive information; it will help summarize it" is a much subtler attack than a buffer overflow, and it is much harder to detect with conventional supply-chain tooling.

Practical mitigations: prefer first-party MCP servers over third-party ones; audit the system prompts of any MCP server you connect, not just its code; assume tool descriptions are part of the model's effective system prompt and review them as such; pin to specific versions of skills and MCP servers rather than tracking "latest"; and limit which agents can call which MCP servers, so that a compromise in one does not propagate to all.

Excessive agency from runtime drift

A particular risk profile of long-running agents is that they accumulate state. The container's filesystem grows; tool results pile up in the context; the agent's sense of what it has accomplished gets ever more divorced from what the user asked. Even without an active attacker, sufficiently long sessions tend toward behavior the original designer did not anticipate — the agent decides to read more files than it needs, makes more tool calls than the task required, "improves" something nobody asked it to improve.

On Managed Agents, this is partially mitigated by the platform's context compaction, which keeps long sessions tractable, and partially by your own session-bounding decisions: how long you let a session run, what you do when token usage exceeds a budget, whether you cap the number of tool calls. Use the usage field on the session object aggressively. Set hard ceilings on session duration and tool-call count. When in doubt, prefer many short sessions over one long session: the smaller blast radius is worth the per-session-hour startup cost.

Oversight gaps from autonomy

Agents run faster than humans review. The CVE monitor in Section 6 produces a daily Markdown file with a few CVEs in it; if it ran across hundreds of teams in your organization, nobody would read every one of them every day. The platform gives you the tools to review (the Console, the event log, the usage statistics) but it does not, today, give you alerting, anomaly detection, or budget enforcement at the org level. Those have to be built on top.

Concretely: pipe session events into your existing observability stack on completion, alert on patterns (excessive tool calls, denied confirmations, failed sessions, token usage above a threshold), define what "normal" looks like for each agent, and treat deviations the way you would treat any other anomaly signal. A failed session that nobody investigates is an exfiltration that nobody detected.

The B0 boundary: what crosses between you and Anthropic

Finally, the boundary that Managed Agents introduces, B0 in Figure 7.1. Everything you put into a `user.message` event crosses this boundary. Everything that comes back out as `agent.message` or

tool_result crosses it the other way. API keys cross it. Session metadata crosses it. The cumulative content of every session you run is on Anthropic's side after it crosses.

The defenses here are mostly contractual and procedural, not technical. Understand what your agreement with Anthropic says about data handling, retention, and incident response. Apply data-classification policies before deciding what content can be processed by Managed Agents at all — some classes of data may simply not be appropriate for any third-party processing. Avoid putting credentials, secrets, or sensitive PII into prompts or tool inputs that will end up in event logs, the same way you would avoid logging them anywhere else.

7.4 A defense-in-depth checklist for Managed Agents deployments

It is worth pulling all of the above into a single checklist that a security architect can run against any proposed Managed Agents deployment. Not all items will be necessary for every agent; the point is that they should be considered.

Agent definition

- System prompt avoids embedding sensitive material that does not need to be there.
- Tool surface is narrowed to the minimum the task requires (do not enable the full agent_toolset_20260401 if you only need web_fetch).
- MCP servers are first-party where possible; third-party servers are pinned and reviewed.
- Skills are pinned, reviewed, and have a known provenance.
- Permission policies require confirmation on actions whose blast radius warrants it (writes, bash, custom tools that take action in your systems).

Environment definition

- Network egress is restricted to a host allow-list. Unrestricted is acceptable only for early prototyping, never production.
- Mounted files are minimal and read-only where possible.
- Pre-installed packages are the smallest set the agent actually needs.
- No secrets are mounted as plain files; if credentials are needed, they are short-lived, scoped, and ideally rotated per session.

Application code

- Inputs to user.message events are sanitized when they come from less-trusted sources.
- Tool-confirmation logic implements an allow-list of expected actions, not a deny-list of known-bad ones.
- Session usage and duration have hard caps; runaway sessions are interrupted.
- Output is treated as untrusted content for downstream consumers (sanitization, validation, integrity checks).

- Custom tool implementations enforce their own input validation; do not rely on the model to send well-formed arguments.

Operational

- API keys are rotated, scoped, and monitored in the Console's usage view.
- Session events are pulled into your own observability stack on completion.
- Alerts are defined for anomalous patterns (excessive tool calls, denied confirmations, failed sessions, token spikes).
- Agent and Environment definitions are version-controlled and code-reviewed before changes.
- Incident response runbooks include "agent has been compromised" as a scenario.

No checklist makes a system secure. But running through one of these forces the conversation to happen in the right places before deployment, which is when you have the most leverage and the least pressure.

7.5 Where the threat model is still genuinely open

I want to close this section by being honest about what we do not yet know. Agentic AI security is young; Managed Agents specifically is months old; the population of attackers paying attention to it is small but growing. There are several questions where the right answer is "we are still figuring it out."

Cross-tenant isolation in the runtime is a question. Anthropic operates many customers' sessions on shared infrastructure; the same trust questions you would ask of any multi-tenant cloud apply, and the answers are mostly behind contractual disclosure rather than direct visibility. The track record will be built up over time.

Multi-agent threat modeling is genuinely open. The orchestrator-worker pattern, available in research preview, multiplies the surfaces we discussed above: each agent is a context, each context is an injection target, and the data flowing between them is in the same kind of trusted-and-untrusted-mixed channel we identified for events. Best practices are still emerging; if you adopt the research-preview features, you are likely to be ahead of the published guidance, and you should treat that as both an opportunity and a risk.

Long-term effects of running many agents at scale are unknown. What does the security posture of an organization look like when there are five thousand agents, not five? How do you do change management on agent fleets? How do regulators come to expect you to demonstrate "we have control of our agents"? None of these questions has a settled answer in April 2026. They will, eventually. For now, the right move is to deploy thoughtfully, document deliberately, and stay engaged with the practitioner community as norms develop.

8. Where Managed Agents fits in your toolbox

We close with a more pragmatic question than the threat model: when is Managed Agents the right tool, and when is something else? The product is genuinely useful; that does not mean it is the right answer to every question. A short tour through where it fits well, where it fits poorly, and what to reach for instead.

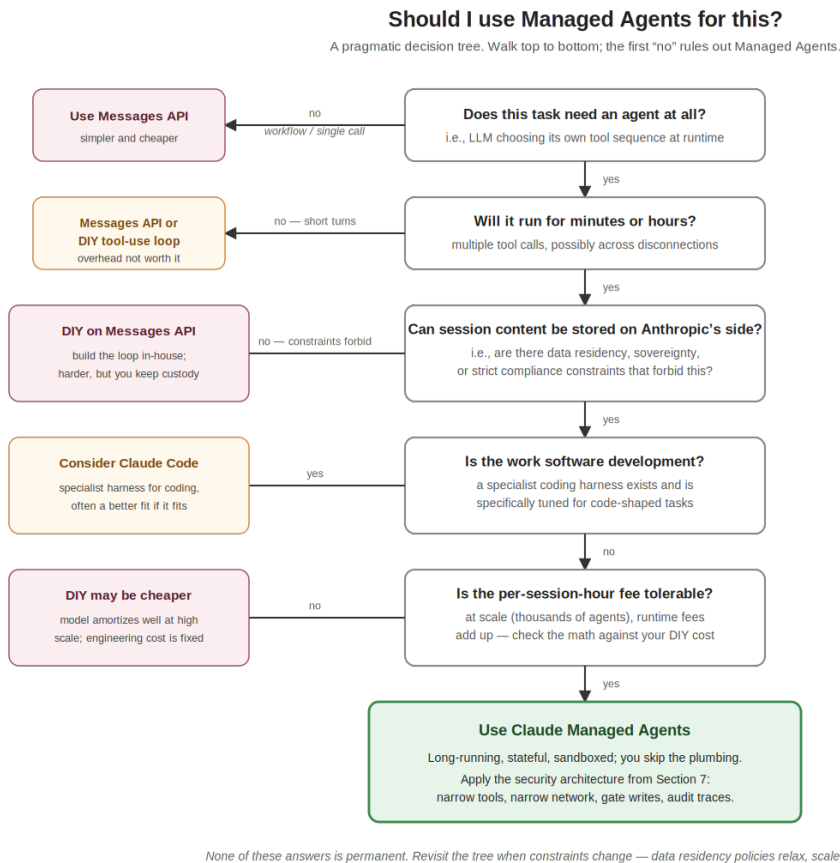


Figure 8.1 — A decision tree for whether to use Managed Agents on a particular task. The first “no” rules it out; revisit when constraints change.

8.1 Where it fits well

Managed Agents is at its best on tasks that have all of the following properties:

- They genuinely benefit from agentic flexibility — the model needs to choose its own steps at runtime, not follow a fixed pipeline.
- They are long-running enough that the per-session-hour fee is amortized over real work — minutes or hours, not seconds.
- They are tool-heavy — the agent makes many calls during the task, and writing the tool execution layer yourself would be repetitive, error-prone work.

- They tolerate the trust placement — the data being processed is acceptable to send to Anthropic, and the operational characteristics of a hosted runtime are acceptable for this workload.
- They benefit from observability — the audit trail in the Console and the event log is genuinely useful for the task, not just nice to have.

The CVE monitor we built in Section 6 fits all five. So does, for example, a quarterly compliance-evidence collector that walks across multiple data sources, a long-running migration assistant that operates over a codebase, a customer-research agent that interviews multiple sources and synthesizes, an incident-triage agent that pulls from logs and runbooks. The common thread is "long, branchy, tool-heavy, and you would build the same plumbing yourself anyway."

8.2 Where it fits poorly

Managed Agents is the wrong tool when:

- The task is short and one-shot. A chat completion or a single-step generation has no reason to pay the per-session-hour overhead. Use the Messages API directly.
- The task is fully predictable. If you know the steps in advance and they do not change, you have a workflow, not an agent. Build it as a deterministic pipeline with LLM calls inside, not as an agent loop.
- Data residency or sovereignty rules forbid sending session content to Anthropic's infrastructure. This is a hard constraint and a relatively common one in regulated industries; if you have it, you have to keep the loop in-house.
- You need full custody of the agent loop for compliance, audit, or operational reasons. This overlaps with the data residency case but is broader — some organizations want to keep the agent loop on their hardware as a matter of principle, even when the data could legally be elsewhere.
- Latency is critical. Managed Agents is optimized for throughput on long-running tasks, not for sub-second turnaround. If you need a snappy user-facing assistant, use the Messages API with your own thin loop.
- You are doing software engineering specifically. Claude Code is a specialist harness for code-shaped tasks and is usually a better fit than a general-purpose agent runtime for that domain.
- Cost-efficiency at very high scale matters more than time-to-deployment. The fixed engineering cost of building your own harness is amortized very well across thousands of agents; the variable per-session-hour cost of Managed Agents is not.

8.3 What to reach for instead

Three alternatives are worth knowing well:

Messages API with your own loop. The most general alternative. You build the loop, the sandbox, the tool execution, and the persistence yourself. Maximum control, maximum responsibility, and the only realistic option when data residency or full-custody constraints rule out the managed runtime. This is what most production agents looked like before Managed Agents existed, and it remains the right answer for many specific cases.

Claude Code. A specialist coding harness. If your task is software engineering — reading codebases, fixing bugs, writing tests, refactoring — Claude Code is purpose-built and likely to outperform a general-purpose Managed Agent on the same task. The trade-off is that Claude Code is opinionated about what it does; if your task is not really software engineering despite involving some code, the specialist tool may fit poorly.

Workflows over agents. The simplest alternative is the one that often gets overlooked: don't use an agent at all. If you can decompose a problem into a fixed pipeline of LLM calls and tools, you almost always should. Workflows are easier to reason about, easier to test, easier to audit, and easier to secure. The "Building Effective Agents" framing is exactly right: agents are a heavy tool, suited to problems that genuinely require runtime flexibility, not the default starting point.

8.4 A closing thought for the security architect

It is tempting, when a new platform like Managed Agents arrives, to either embrace it everywhere or refuse it everywhere. Neither is quite right. The platform is genuinely well-designed, the abstractions are durable, and many of the security primitives — the egress allow-list, the tool-confirmation policy, the session trace — are exactly the right shapes. At the same time, the threat model is real, the trust placement is significant, and the new attack surfaces are not theoretical.

The pattern that consistently produces good outcomes in this kind of situation is: adopt deliberately, in narrow scopes, with the security architecture made explicit before the first agent ships. Build a few small, well-constrained agents on Managed Agents (the CVE monitor in Section 6 is a good template). Run them long enough to develop institutional muscle around their operational and security characteristics. Use the lessons to shape your organization's broader policy on agentic AI. Then, with that foundation, broaden the scope as confidence grows and as the practitioner community's shared knowledge matures.

What you should not do, in either direction, is decide based on enthusiasm or anxiety. The shape of the threat model has been laid out in Section 7; the shape of the use cases has been laid out in this section; the operational mechanics have been laid out in the earlier sections. Whatever you decide for your organization, decide it knowing what you are deciding for and against. That is what the rest of the work in this tutorial was for.

Appendices

Five appendices follow. Appendix A is a set of one-page reference cards for the four core abstractions. Appendix B is a glossary of the conceptual vocabulary used in this tutorial. Appendix C is a short list of further reading. Appendix D is a quick-reference table of acronyms and short technical terms. Appendix E is an extended definition of the term "harness" and its relationship to the meta-harness pattern that organizes the platform.

Appendix A — Reference cards for the four abstractions

Each of the four core concepts in Managed Agents — Agent, Environment, Session, Events — gets a one-page reference card. These are deliberately compact: enough to remind you of the shape, not so much as to duplicate the documentation.

Agent

A reusable definition. Inert until instantiated by a Session.

What it specifies

- Model (e.g., claude-opus-4-7)
- System prompt (role, rules, constraints, workflow)
- Tools (built-in bundle or curated subset)
- MCP servers (your own or third-party tool providers)
- Skills (reusable capability bundles)
- Permission policies (which tool calls require confirmation)

How it is created

- CLI: `ant beta:agents create --from agent.yaml`
- SDK: `client.beta.agents.create(...)`
- Reusable across many sessions; referenced by ID

Security relevance

- Where most offensive-security choices live: tool surface, MCP server selection, permission policies
- Treat as security-critical configuration: version, review, gate behind approval
- Tool descriptions are part of the effective system prompt; review them too

Environment

A container template. The security primitive of the platform.

What it specifies

- Base image and pre-installed packages
- Network egress policy (unrestricted, or an allow-list of hosts)
- Mounted files (read-only static assets the agent needs)
- Environment variables (including, with care, credentials)

How it is created

- CLI: `ant beta:environments create --from env.yaml`
- SDK: `client.beta.environments.create(...)`
- Reusable; one Agent can be paired with several Environments and vice versa

Security relevance

- The single most load-bearing security control in the platform
- Network allow-list bounds where data can flow regardless of what the model decides
- Mounted files and packages should be the minimum the agent actually needs
- Avoid mounting credentials as plain files; prefer short-lived scoped credentials

Session

A running instance: an Agent executing inside an Environment toward a goal.

Properties

- Stateful: persistent filesystem in the container, durable event log
- Long-lived: minutes to hours; survives client disconnections
- Steerable: additional user events can redirect or interrupt mid-execution
- Observable: every event is in the trace, retrievable through API or Console

How it is created

- SDK: `client.beta.sessions.create(agent=AGENT_ID, environment_id=ENV_ID)`
- Returns a session ID; container is provisioned but idle until first event
- Costs: standard token costs plus per-session-hour runtime fee while active

Security relevance

- The unit of blast radius: harm from a successful injection plays out within one session
- Hard caps on duration and tool count limit runaway sessions
- Many short sessions are usually safer than one long session
- Session events are stored in Anthropic's database; classify accordingly

Events

The append-only log of everything that happened in a session.

Event types (selection)

- `user.message` — input from your application
- `user.tool_confirmation` — allow/deny decision on a gated tool call
- `user.custom_tool_result` — result from a tool you executed in your code
- `agent.message` — model-generated text output
- `agent.tool_use` — model-requested tool invocation
- `agent.custom_tool_use` — model-requested invocation of a custom tool
- `session.status_idle` — lifecycle event; check `stop_reason` for next step

How they flow

- You send via `client.beta.sessions.events.send(...)`
- You stream via `client.beta.sessions.events.stream(...)` (SSE under the hood)
- Application logic dispatches on `event.type` — the canonical pattern

Security relevance

- The central artifact: model reads from it, runtime writes to it, your app does both
- No architectural separation between trusted and untrusted content within the log
- Pull events into your own logging on completion for retention and audit
- Treat anything that entered as a tool result as untrusted on subsequent reads

Appendix B — Glossary

The terms below are used throughout this tutorial. Some are specific to Managed Agents; some are general agentic-AI vocabulary. Definitions are concise and aimed at how the term is used in this document, not exhaustive. For acronyms and short technical terms, see Appendix D.

Agent — In Managed Agents specifically: a reusable definition (model, prompt, tools, MCP servers, skills) that, when paired with an Environment in a Session, becomes a running agent. In general agentic-AI usage: a system where an LLM dynamically directs its own process and tool usage in pursuit of a goal.

Agent loop — The control flow that calls the model, parses its output, executes any requested tools, appends results to the context, and repeats. In Managed Agents, the loop runs inside Anthropic's runtime; in DIY architectures, you write it yourself.

ant — The Managed Agents command-line tool. Its commands take the form `ant beta:<resource> <action>`.

Beta header — The HTTP header `anthropic-beta: managed-agents-2026-04-01` required on all Managed Agents API calls during the beta period. Set automatically by the SDK and CLI.

Compaction — The process of summarizing or shrinking older parts of a long-running session's context to keep it within the model's context window. In Managed Agents, this is handled by the runtime.

Context window — The maximum amount of text the model can attend to in a single inference call. Includes the system prompt, the goal, prior tool results, retrieved content, and the agent's own prior reasoning.

Custom tool — A tool whose execution happens in your code rather than in the sandbox. The runtime emits `agent.custom_tool_use`; you process and reply with `user.custom_tool_result`.

Environment — In Managed Agents specifically: a reusable container template (image, packages, network rules, mounted files). The security primitive of the platform.

Event — A single entry in the session's append-only log. Has a type (`user.message`, `agent.tool_use`, etc.) and content specific to the type.

Event log — The full append-only history of events in a session. Stored durably by Anthropic; readable via the API or the Console.

Harness — The software wrapping around an LLM that turns it into an agent: the loop, the tool execution layer, the sandbox, the memory. Managed Agents is sometimes called a "meta-harness" because it is general enough to host many specific harnesses. See Appendix E for an extended treatment.

Meta-harness — A harness designed to be general enough to host many possible specific harnesses underneath, by exposing stable interfaces (Agent, Environment, Session, Events) above an evolving implementation. The architectural pattern Managed Agents is built on. See Appendix E.

MCP — Model Context Protocol — an open protocol for exposing tools and resources to LLMs. MCP servers can be connected to a Managed Agents Agent definition. (See Appendix D for the acronym expansion.)

Permission policy — The Agent-level configuration that determines which tool calls require confirmation before execution. Currently supports `requires_confirmation` gating.

Prompt injection — A class of attack where adversarial text in content the model reads (a tool result, a retrieved document, a user input) overrides the agent’s intended behavior.

ReAct — A reasoning pattern that interleaves thought, action, and observation in a single agent loop. The default style of most modern tool-using agents.

Session — In Managed Agents specifically: a running instance of an Agent inside an Environment, executing toward a goal and producing an event log. Long-lived, stateful, observable.

Skill — A reusable bundle of instructions (and optionally code and resources) that extends what an agent knows how to do, without bloating the system prompt.

SSE — Server-Sent Events — the HTTP-based one-way streaming protocol used to deliver events from the runtime to your application as they happen. The client opens an HTTP connection; the server pushes events down it as they occur. Resumable via cursor; survives client disconnections. See Appendix D for the acronym expansion.

Stop reason — A value on a `session.status_idle` event indicating why the loop paused: `end_turn` (the agent finished), `requires_action` (a confirmation is pending), and others.

Tool — A function the agent can invoke. Defined by name, description, and parameter schema. The model chooses when to call which tool; the runtime executes the call.

Tool confirmation — The mechanism by which gated tool calls (those marked `requires_confirmation`) are paused for application-level approval before execution.

Trust boundary — A line in an architecture across which entities have different privileges or trust levels. In Managed Agents, the four boundaries B0–B3 in Figure 7.1 are the principal ones.

Workflow — An LLM-using system whose control flow is pre-defined in code, in contrast to an agent whose control flow is decided by the model at runtime. Usually the right starting point if the steps are knowable in advance.

Appendix C — Further reading

A short, opinionated list of things worth reading next, organized by topic and weighted toward conceptual depth. This is not a comprehensive bibliography; it is a curated set of starting points that complement the material in this tutorial.

Foundational concepts

Anthropic, "Building Effective Agents." The single best starting point for the workflow-vs-agent distinction and the design philosophy of "keep it simple." Already in your Project Knowledge; worth re-reading after this tutorial because the framing will land differently with the platform context now in place.

Anthropic Engineering Blog, "Scaling Managed Agents: Decoupling the brain from the harness." The deepest published explanation of the meta-harness concept, the OS-virtualization analogy, and the design decisions behind the four core interfaces. Strongly recommended; it will deepen everything in Sections 1–3 of this tutorial.

Platform documentation

Claude Managed Agents documentation (platform.claude.com/docs). The authoritative source for syntax, API reference, and current capabilities. Treat this tutorial as the conceptual scaffolding and the docs as the ground truth on specifics. Both will evolve; the docs will evolve faster.

Model Context Protocol (modelcontextprotocol.io). If you adopt Managed Agents at any scale, you will end up working with MCP. The protocol's own documentation is the right reference, especially for the parts of the threat model that involve MCP server selection and trust.

Security and threat modeling

OWASP Top 10 for LLM Applications. A practitioner-oriented enumeration of the principal LLM security risks. Maps imperfectly to agentic systems specifically, but the categories — prompt injection, insecure output handling, training data poisoning, supply chain — line up well enough that it is a useful reference when you are building a threat model from scratch.

Simon Willison's writing on prompt injection. A long-running practitioner perspective on the prompt injection problem, including the running argument that there is no clean technical solution at the model layer alone. Useful for calibrating expectations about what defenses are realistic and what defenses are wishful thinking.

Adjacent material in your Project Knowledge

Several of the resources already in your Project Knowledge will read differently after this tutorial; it is worth revisiting them:

- **Your agentic_ai_answers.md** Q11 on agent failure modes will map onto Section 7 here almost item-by-item. The list of failure modes there is the conceptual scaffolding; this tutorial provides the platform-specific instantiation.
- **Your agentic_ai_conversation.docx** on the invocation flow lays out the conceptual loop — LLM proposes, runtime disposes — that is the basis of everything Managed Agents productizes. Worth re-reading with the four abstractions in mind.
- **Anthropic's 2026 Agentic Coding Trends Report** makes the argument for "embedding security architecture as a core design principle from project inception rather than retrofitting it." Section 7 of this tutorial is what that looks like in practice for Managed Agents.

A final framing note

The single most useful thing you can do after finishing this tutorial is build a small, low-stakes Managed Agent and run it for a few weeks. The CVE monitor in Section 6 is designed exactly for that purpose. Reading about agents, even carefully, develops only a thin sense of what they are actually like to operate. Running one — watching it succeed, watching it fail, watching the Console fill up with traces, watching your own intuitions about what to gate and what to permit shift over time — builds the kind of understanding that no amount of reading produces. The tutorial is preparation. The actual learning happens after.

Appendix D — Terms and abbreviations

A quick-reference table of acronyms and short technical terms used in this tutorial. Where an entry has a longer conceptual treatment elsewhere, that location is noted.

Term	Expansion / definition
API	Application Programming Interface. The general term for a defined interface between two software components; in this document, usually the Anthropic API specifically.
BCP/DR	Business Continuity Planning / Disaster Recovery. The standard category of preparedness for service outages and data loss; relevant when treating Anthropic as a critical SaaS dependency (Section 7.1).
CISA	Cybersecurity and Infrastructure Security Agency (United States). Publishes the Known Exploited Vulnerabilities (KEV) catalog and other security advisories.
CLI	Command-Line Interface. In this document, usually the ant tool (Sections 4.1, 5).
CSV	Comma-Separated Values. A common tabular data format.
CVE	Common Vulnerabilities and Exposures. The standard identifier scheme for publicly known security vulnerabilities.
CVSS	Common Vulnerability Scoring System. The standard severity-scoring framework for CVEs; v3.1 is current as of writing.
IAM	Identity and Access Management. The standard category of cloud-provider controls for who can do what.
JSON	JavaScript Object Notation. A common data interchange format.
KEV	Known Exploited Vulnerabilities catalog (maintained by CISA). A curated list of CVEs known to be actively exploited in the wild.
LLM	Large Language Model. The category of model that Claude is an instance of.
MCP	Model Context Protocol. An open protocol for exposing tools and resources to LLMs. See also the glossary entry in Appendix B.
NVD	National Vulnerability Database. The U.S. government repository for cybersecurity vulnerability intelligence; the canonical source for CVE data and the primary feed used by the CVE monitor in Section 6.
OAuth	An open standard for authorization, often used for delegated access in cross-service integrations.
OIDC	OpenID Connect. An identity layer on top of OAuth 2.0; used in federated authentication patterns including the AWS role-assumption pattern referenced earlier in this project.

Term	Expansion / definition
OWASP	Open Worldwide Application Security Project. A practitioner community that publishes widely used security guidance, including the Top 10 for LLM Applications referenced in Appendix C.
PII	Personally Identifiable Information. Data that can identify a specific individual; relevant when classifying what content should and should not flow into a Managed Agents session.
SaaS	Software as a Service. The category of hosted-software product that Managed Agents falls into.
SDK	Software Development Kit. In this document, usually the Anthropic Python SDK specifically (Sections 4.2, 5, 6).
SIEM	Security Information and Event Management. The standard category of platform for centralized security log collection, correlation, and alerting.
SSE	Server-Sent Events. The HTTP-based one-way streaming protocol used to deliver events from the runtime to your application. See also the glossary entry in Appendix B.
SSO	Single Sign-On. An authentication pattern that lets a user sign into one identity provider and access many services without re-authenticating.
STS	Security Token Service (in AWS specifically). Issues short-lived credentials in response to role-assumption requests; relevant to the credential-handling pattern referenced earlier in this project.
YAML	YAML Ain't Markup Language. A human-readable data serialization format, used in this tutorial for Agent and Environment definitions (Section 6).

Appendix E — Harness: an extended definition

The term "harness" appears throughout this tutorial without ever getting a careful definition in the body. This appendix is that definition. It expands on the brief glossary entry in Appendix B with the full treatment, including the meta-harness pattern that is central to how Managed Agents is designed and the implications for how the trust model is drawn.

What is a harness?

A harness is the software that wraps around an LLM and turns it from a text-generating model into a working agent. The model on its own can only do one thing: take a context window in and emit text out. That is not enough to be useful for anything beyond a chat reply. To make the model do real work — call tools, persist state, recover from errors, decide when to stop — you need a layer of code around it that handles all the things the model itself cannot do. That layer is the harness.

Concretely, a harness typically owns:

- **The agent loop** — the control flow that calls the model, parses what comes back, dispatches any tool calls the model requested, captures the results, appends them to the context, and calls the model again.
- **Tool execution** — the machinery that actually runs `web_fetch` or `bash` or whatever, taking the model's structured request and turning it into a real action.
- **The sandbox** — wherever the tools run, with whatever isolation and resource limits apply.
- **Memory and state** — persistent filesystem, conversation history, anything that needs to outlive a single model call.
- **Context management** — assembling what goes into each model call, compacting when it grows too large, deciding what to keep and what to drop.
- **Error handling and recovery** — retrying transient failures, recovering from network drops, handling malformed model output.

If you build an agent on the Messages API yourself, you write the harness. Every line of it is your code, running on your infrastructure. That gives you maximum control and maximum responsibility. The Messages API itself is *not* a harness — it is just the interface to the model. A harness is what you wrap around it.

What is a meta-harness?

The thing that makes Managed Agents distinctive, and that the Anthropic engineering team is explicit about, is that the platform is what they call a meta-harness. The idea is that any specific harness encodes assumptions about what the model can and cannot do — assumptions that go stale as models improve.

Their example: an older harness added "context resets" to handle a behavior in Claude Sonnet 4.5 where the model would wrap up tasks prematurely as it sensed its context limit approaching. When they tested the same harness on Claude Opus 4.5, the behavior was gone, and the workaround was no longer needed. The harness had encoded an assumption that had become wrong.

The meta-harness solution is to design a system that is general enough to host many possible specific harnesses — opinionated about the interfaces (the four abstractions: Agent, Environment, Session, Events) but unopinionated about the harness implementation underneath. Anthropic runs a particular harness today; in two years they will likely run a different one tuned for whatever the model has gotten better at; the four interfaces around them stay stable. That is the analogy to operating system primitives that comes up in their writing — the `read()` system call has outlasted decades of evolving disk hardware because it abstracts the right thing.

Related distinctions

Harness vs. agent. The harness is the wrapping; the agent is the running thing the harness produces. "Claude Code" is a harness — a specialist one tuned for software engineering. A Managed Agents agent is a generic harness configured for a particular task.

Harness vs. workflow. A workflow is a fixed pipeline of LLM calls written in code by a developer; the orchestration is yours, the LLM is just a component. A harness gives the orchestration to the LLM at runtime, which is what makes the resulting system an agent. The harness is the machinery; the workflow-vs-agent distinction is about who decides the next step.

The shift Managed Agents represents. Adopting Managed Agents is, at the most fundamental level, the choice to stop owning the harness. Most of what Section 7 calls out as "moved to Anthropic's trust boundary" — the loop, the sandbox, the tool execution layer, session storage — is the harness. What stays with you is everything outside the harness: the agent definition, the user goals you submit, the events you stream back, the application logic on top.

The framing that matters most for a security architect

When Section 7 talks about what crossed the B0 boundary into Anthropic's responsibility, what crossed is the harness. The harness used to be your code; now it is their code. Everything that follows in the threat model is consequences of that choice.